IMPLEMENTATION MANUAL

Title:

Net Builder -Interactive Network Topology Simulator

A. Chethana Sri - 2024503524 -chethanasriadusumalli@gmail.com

Contents

Chapter	Section
1	Project Overview
2	System Architecture and Connectivity
3	Implementation Workflow
4	Dependencies and Hardware Requirements
5	Configuration Reference
6	Installation Procedure
7	Execution Procedure
8	Verification and Test Plan
9	Project Structure and Responsibilities
10	Troubleshooting
11	Scope, Limitations, Future Work
12	Conclusion

Problem Statement – Key Points

Computer networking is an essential part of computer science, but understanding how network devices communicate and how data packets travel between them can be challenging for students. Traditional learning methods often depend on theoretical explanations and static network diagrams, making it difficult to visualize network topologies, understand device connectivity, and observe network behavior in practice.

Existing network simulation tools may require extensive configuration and prior technical knowledge, creating difficulties for beginners who want to experiment with different network structures. Furthermore, designing a network topology and verifying whether its devices can communicate are two distinct tasks that require both a visual representation and an underlying network simulation environment.

To address these challenges, **NetBuilder – Interactive Network Topology Simulator** aims to provide a web-based platform for creating, editing, and simulating network topologies through an interactive graphical interface. The system allows users to add network devices such as computers, switches, routers, and servers, establish connections between them, and modify the topology using drag-and-drop operations. By integrating a React-based frontend with a Python Flask backend and Mininet, the project connects visual topology design with network emulation and connectivity testing for supported devices.

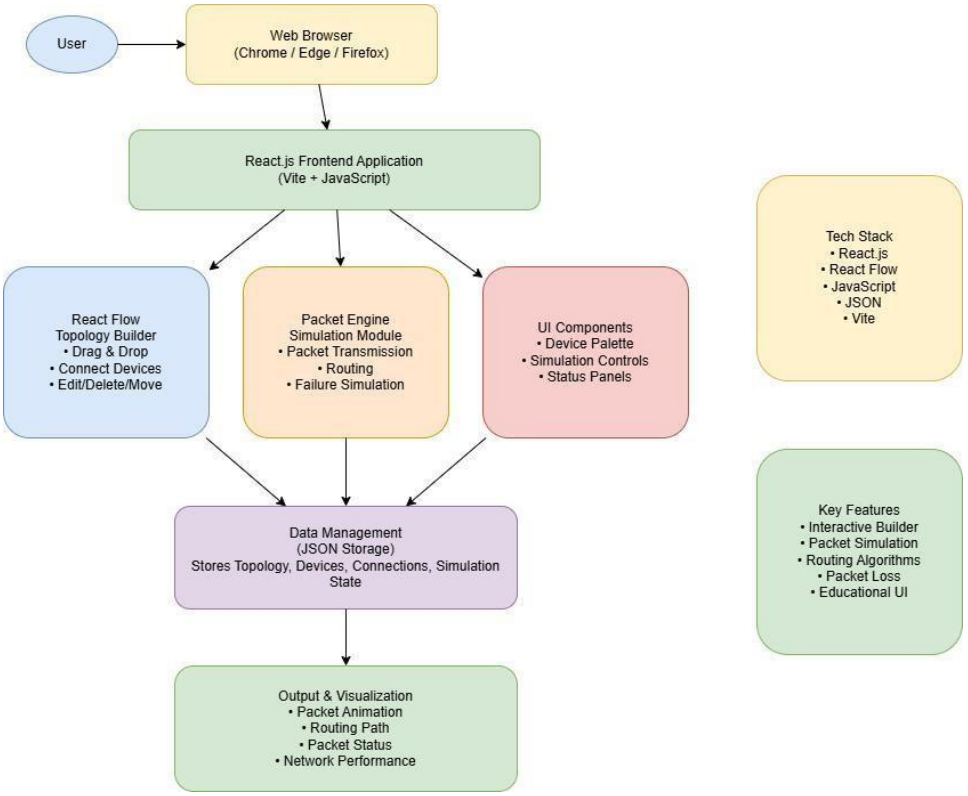
The primary objective of NetBuilder is to simplify networking experimentation by providing an accessible environment for topology construction, simulation control, and observation of network connectivity. The platform aims to help students understand networking concepts through practical experimentation rather than relying solely on theoretical learning. It also provides a foundation for future enhancements involving packet-level visualization, traffic analysis, performance measurement, and advanced network configuration.

Additionally, NetBuilder is designed to provide a modular and extensible architecture that supports the development of more advanced networking features. The integration of React Flow for topology visualization, Flask for backend communication, and Mininet for network emulation creates a foundation for connecting graphical network designs with practical testing environments. Users can experiment with different network arrangements, initiate simulation operations, and check connectivity through the backend. The project also aims to improve the learning experience by making network configuration and testing more interactive, understandable, and accessible. In the future, the platform can be extended to support detailed packet movement visualization, network traffic monitoring, latency and throughput analysis, and performance evaluation under different network conditions, enabling a more comprehensive understanding of real-world networking concepts.

System Architecture:

The NetBuilder – Interactive Network Topology Simulator follows a modular architecture that enables users to design, simulate, and visualise computer networks through an interactive web interface. The application is developed using React.js and React Flow, allowing users to create network topologies using drag-and-drop functionality.

The topology builder interacts with the packet simulation module to animate packet transmission between networking devices such as Computers, Switches, Routers, and Servers. The simulator also supports features like packet loss, latency, congestion, and alternate routing to demonstrate real-world network behaviour. Network topology data and simulation information are stored in JSON format, and the results are displayed through a user-friendly interface with real-time packet status and routing visualisation.



Proposed Solution

Month 1: Foundation and Setup

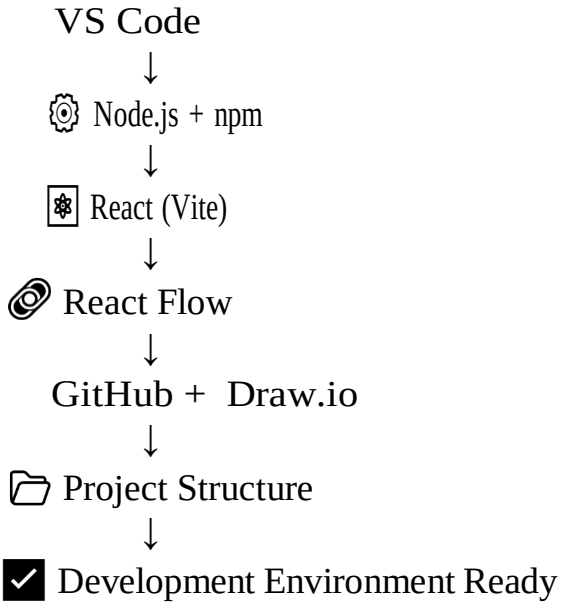
Week 1–2: Literature Review

- During the literature review, we will study the fundamentals of computer networking, including LAN, WAN, network topologies, and communication models, to understand how devices communicate and exchange data.
- We will analyze the working principles of networking devices such as Computers, Switches, Routers, Servers, Hubs, and Network Interface Cards (NICs), along with packet transmission, packet forwarding, switching techniques, and routing algorithms.
- We will explore popular network simulation tools like Cisco Packet Tracer, GNS3, and NS-3 to understand their features, advantages, and limitations. We will also study React Flow, a node-based visualization library, for implementing drag-and-drop network topology creation.
- Additionally, we will review IEEE and ACM research papers on network visualization, packet animation, and interactive learning platforms. Finally, we will study real-world networking scenarios such as client-server communication, packet loss, congestion, routing, and network failures, which will help us design an educational and interactive network topology simulator.
- Study the fundamentals of computer networking, including LAN, WAN, MAN, PAN, network topologies (Bus, Star, Ring, Mesh, Tree), network architectures (Client-Server and Peer-to-Peer), and the OSI and TCP/IP models. This will provide a strong understanding of how data is transmitted between devices across different types of networks.
- Analyze the working principles of networking devices such as Computers, Switches, Routers, Servers, Hubs, Bridges, Repeaters, Gateways, and Network Interface Cards (NICs). Study packet forwarding, MAC addressing, IP addressing, routing protocols, switching techniques, packet encapsulation, and packet transmission to understand how communication occurs within a computer network.
- Explore popular network simulation and visualization tools such as Cisco Packet Tracer, GNS3, NS-3, Mininet, and Wireshark. Compare their features, advantages, limitations, system requirements, and educational value. Study React Flow as a node-based visualization library for implementing drag-and-drop network topology creation and interactive graphical interfaces.
- Review research papers from IEEE, ACM Digital Library, Springer, and Elsevier on topics such as network visualization, packet animation, educational simulators, virtual networking laboratories, interactive learning systems, network routing visualization, and simulation-based learning. Analyze the methodologies used by existing research and identify features that can improve the proposed project.

Week 3: Tool Installation

- Install **Visual Studio Code** and configure the project development environment.
- Install **Node.js** and **npm** for React application development.
- Create a React project using **Vite**.
- Install and configure the **React Flow** library for drag-and-drop topology creation.
- Set up **Git** and **GitHub** for version control and project management.
- Use **Draw.io** to design system architecture and workflow diagrams.
- Organize the project structure, including reusable React components, CSS files, and routing.
- Design networking device icons and graphical assets for the simulator interface.
- Verify the development environment by running a basic React application integrated with React Flow.

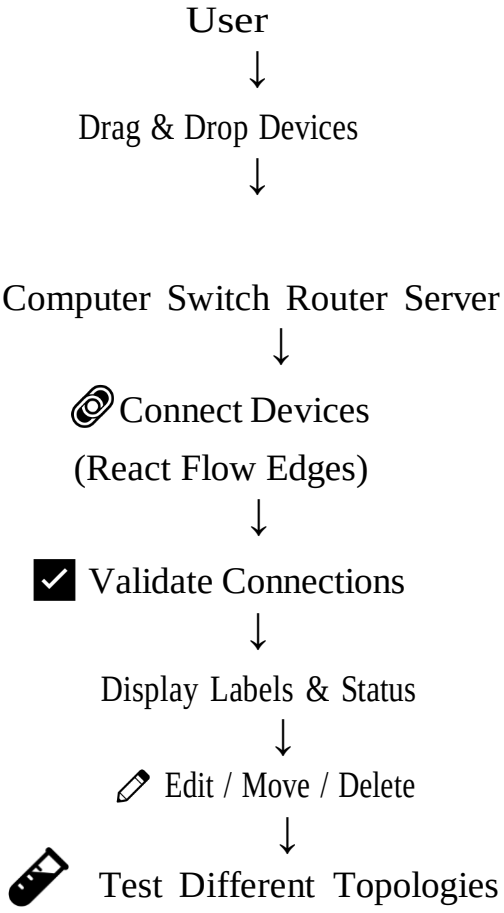
Development Environment Setup Workflow



Week 4: Basic Network Implementation

- Design an interactive workspace for creating network topologies.
- Implement drag-and-drop functionality to add networking devices such as **Computers, Switches, Routers, and Servers**.
- Develop reusable components for each networking device.
- Enable virtual cable connections between devices using **React Flow**.
- Validate connections to ensure proper topology formation.
- Display device labels, icons, and connection status dynamically.
- Implement topology editing features such as adding, deleting, and repositioning devices.
- Test the topology builder using different network layouts to ensure proper functionality.

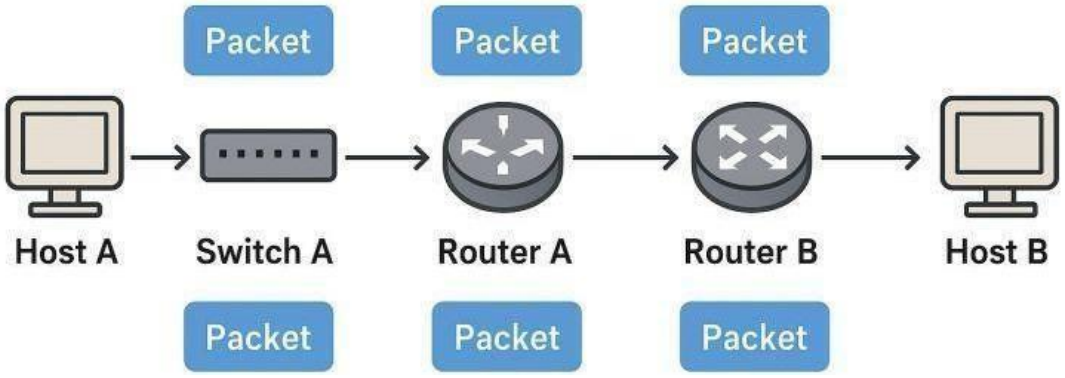
Basic Network Implementation



Month 2: Development and Implementation

Week 5–6: Self-Healing Network

- Design packet objects containing the source address, destination address, packet status, and routing information.
- Develop real-time packet animation to visualize communication between networking devices.
- Implement packet transmission from the source computer to the destination server through intermediate devices.
- Display the packet traversal path using animation.
- Implement packet forwarding based on the created network topology.
- Display packet status messages such as **"Packet Created," "Packet Forwarded,"** and **"Packet Delivered."**
- Validate successful packet transmission under different network configurations.
- Record packet transmission details for analysis and debugging..



Week 7: Networking Device Behaviour

- Implement **Computer** node functionality for generating and sending network requests.
- Develop **Switch** logic to identify connected devices and forward packets appropriately.
- Implement **Router** functionality to determine the appropriate path for packet forwarding.
- Develop **Server** functionality to receive requests and send responses back to the source.
- Display status messages whenever a packet reaches a networking device.
- Visualize packet forwarding decisions using graphical animations.
- Verify correct communication between networking devices.
- Test different communication scenarios involving multiple switches and routers.

Week 8: Advanced Network Simulation Features

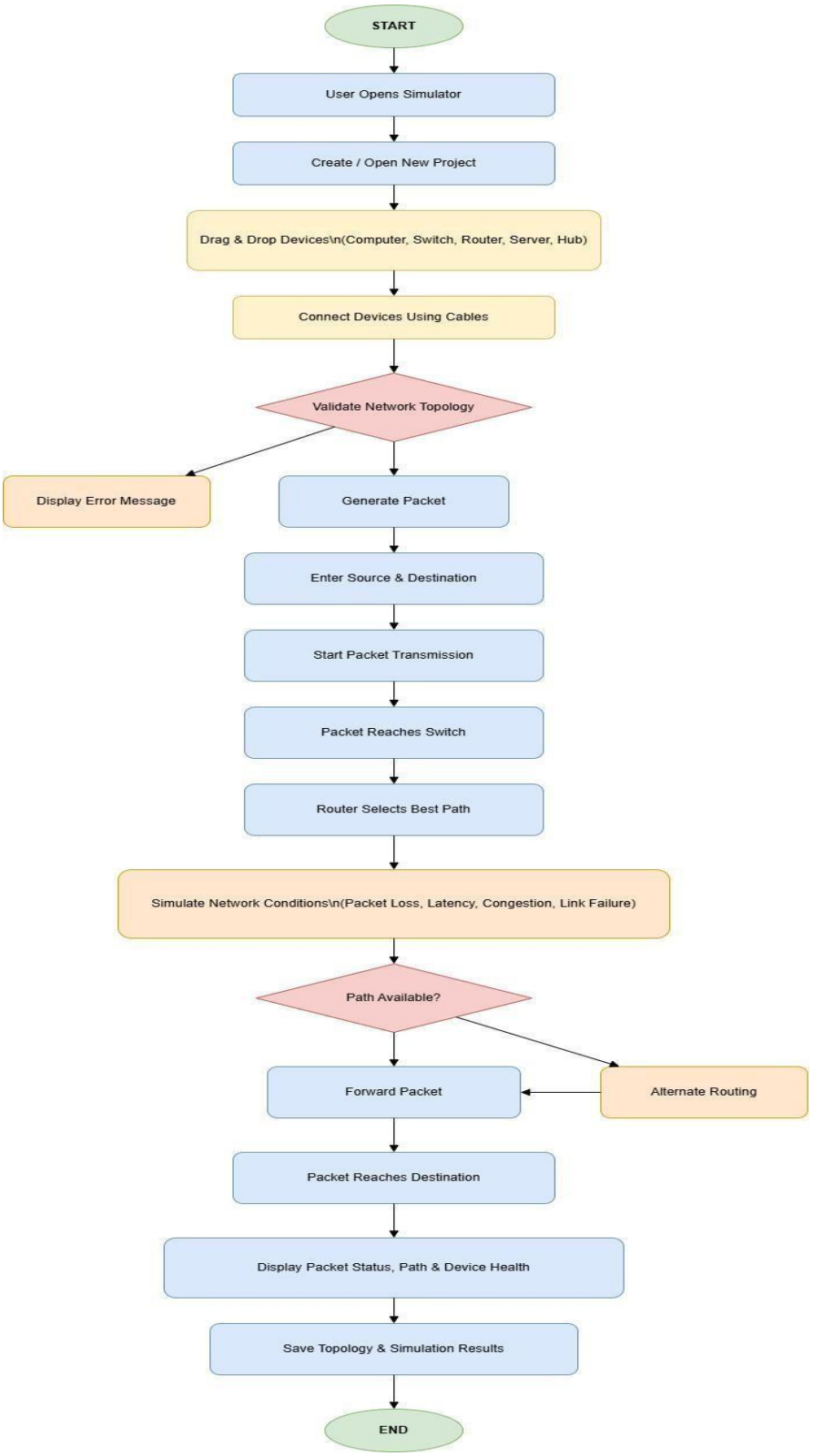
- Implement **packet loss simulation** to demonstrate unsuccessful packet delivery.
- Develop **latency simulation** by introducing configurable transmission delays.
- Simulate **network congestion** by reducing packet transmission speed during heavy traffic.
- Implement **Health Indicators** to display the operational status of networking devices.
- Develop **Packet Speed Control** to allow users to adjust packet animation speed.
- Implement **Alternate Routing** to redirect packets when the primary path becomes unavailable.
- Display warning messages such as **"Destination Unreachable," "Packet Lost,"** and **"No Router Found."**
- Validate all advanced features using different network topologies and failure scenarios.

Month 3: Dashboard, Testing and Documentation

Week 9–10: User Interface, Testing and Performance Evaluation

- Develop a user-friendly dashboard for creating and managing network topologies.
- Integrate the topology builder with the packet simulation module.
- Display real-time packet transmission status and routing paths.
- Perform functional testing on different network topologies.
- Verify packet transmission under normal communication conditions.
- Test network failure scenarios such as broken links, disconnected devices, and packet loss.
- Evaluate packet loss, latency, congestion, alternate routing, and packet speed control features.

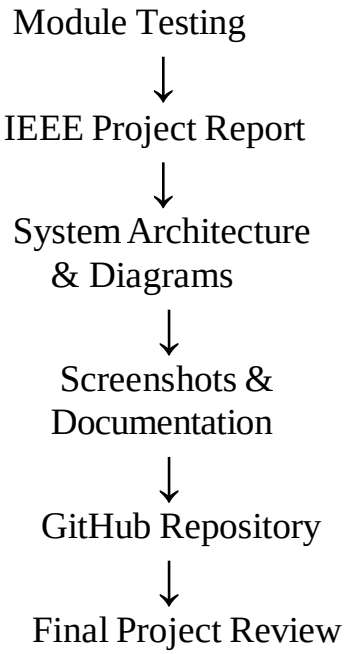
FLOWCHART


START
User Opens Simulator
Create / Open New Project
Drag & Drop Devices\n(Computer, Switch, Router, Server, Hub)
Connect Devices Using Cables
Validate Network Topology
Display Error Message
Generate Packet
Enter Source & Destination
Start Packet Transmission
Packet Reaches Switch
Router Selects Best Path
Simulate Network Conditions\n(Packet Loss, Latency, Congestion, Link Failure)
Path Available?
Forward Packet
Alternate Routing
Packet Reaches Destination
Display Packet Status, Path & Device Health
Save Topology & Simulation Results
END

Week 11: Documentation and Project Preparation

- Perform complete testing of all implemented modules.
- Prepare the project report in **IEEE conference paper format**.
- Design the complete **system architecture** using Draw.io.
- Create workflow and network topology diagrams.
- Capture screenshots demonstrating topology creation, packet transmission, routing, and failure scenarios.
- Organize the source code and upload it to **GitHub** with proper documentation.
- Prepare the final PowerPoint presentation covering the problem statement, objectives, implementation, results, and future scope.
- view all project documents and verify that the project requirements are satisfied

Documentation Workflow



Week 12: Final Demonstration

- Perform complete system testing to ensure all modules function correctly.
- Demonstrate the drag-and-drop network topology creation process.
- Demonstrate real-time packet transmission between networking devices.
- Showcase the functionality of **Computers, Switches, Routers, and Servers** during communication.
- Demonstrate packet loss simulation, latency simulation, congestion visualization, alternate routing, and health indicators.
- Validate different network scenarios, including successful communication and network failures.
- Submit the complete source code, project report, architecture diagrams, presentation slides, and GitHub repository.
- Conduct the final project presentation and explain the educational benefits of the simulator.

Tools and Technologies

- **React.js** – Used to build a dynamic and interactive user interface for the network topology simulator.
- **React Flow** – Enables drag-and-drop creation of network topologies and device connections.
- **JavaScript (ES6)** – Implements the application logic, packet simulation, and user interactions.
- **HTML5** – Provides the basic structure and layout of the web application.
- **CSS3** – Styles the user interface and ensures a responsive, visually appealing design.
- **Node.js** – Provides the runtime environment for developing and running the application.
- **Visual Studio Code** – Used as the primary code editor for development.
- **GitHub** – Manages version control and stores the project source code.
- **Draw.io** – Creates system architecture diagrams, workflow diagrams, and network illustrations.
- **Figma** – Designs the user interface and graphical components before implementation.
- **JSON** – Stores network topology data and application configurations.
- **npm (Node Package Manager)** – Installs and manages project dependencies and libraries.

Expected Outcomes

- Develop an interactive drag-and-drop network topology simulator for creating virtual computer networks.
- Visualize real-time packet transmission between networking devices.
- Improve understanding of packet forwarding, switching, and routing concepts.
- Demonstrate the functionality of Computers, Switches, Routers, and Servers through interactive simulation.
- Simulate real-world network scenarios such as packet loss, latency, congestion, and alternate routing.
- Provide a simple and user-friendly learning platform for beginners in computer networking.
- Enhance practical learning through graphical visualization instead of static diagrams.
- Help students and educators gain a better understanding of networking concepts through hands-on interaction.

References

1) Kurose, J. F., & Ross, K. W., *Computer Networking: A Top-Down Approach*, 8th Edition, Pearson, 2021.

2) Tanenbaum, A. S., & Wetherall, D. J., *Computer Networks*, 6th Edition, Pearson, 2021.

3) React Flow Documentation – https://reactflow.dev

4) Cisco Networking Academy, *Introduction to Networks (ITN)*.

5) IEEE Communications Surveys & Tutorials – Research papers on Network Visualization and Educational Technologies.

6) ACM Digital Library – Research papers on Interactive Learning Systems for Computer Networks.

7) RFC 791 – *Internet Protocol (IP)*, Internet Engineering Task Force (IETF).

8) RFC 793 – *Transmission Control Protocol (TCP)*, Internet Engineering Task Force (IETF).

9) RFC 768 – *User Datagram Protocol (UDP)*, Internet Engineering Task Force (IETF).

10) Draw.io Documentation – https://www.drawio.com